

Code review — academy-hub-api

Runde de review pe proiectul tău, cea mai nouă sus.

Runda 3 — 2026-09-23 — sha 62915d4

Scop: stratul REST nou adăugat (4 controllere) + starea celor 7  din runda 2.

Proiectul **compilează** (`mvn compile` ) și **pornește** . Toate constatările de mai jos au fost reproduse prin rulare: copie în afara repo-ului, H2 în loc de MySQL, aplicația pornită pe portul 8094 și **20 de cereri HTTP reale**. Codul din repo nu a fost modificat.

Ce s-a închis

- **Ai pus `@EntityGraph` pe toate cele patru `findAll()`** — fără să-ți cer eu asta aici. Măsurat: `GET /api/students`, `/api/courses`, `/api/enrollments` = **1 interogare fiecare**, cu tot cu numele cursurilor și al studenților. Regula din company-management ai mutat-o singur în alt proiect. Asta e exact saltul de la „repar linia” la „aplic regula”.
- `Course.enrollments` inițializat (runda 2, B1) 
- `StudentSummary.getId()` trecut pe `UUID` (runda 2, B2) 
- `@NamedEntityGraph("Student.withBooks")` există acum pe entitate (runda 2, B3) 
- `hibernate.dialect` hardcodat — scos din `application.yaml` 
- `spring-boot-starter-web` + `controllers/` adăugate (cerute la runda 1 și 2) 

Critice

B1 — Pe o bază de date goală, TOATE cele patru GET-uri răspund 500

`book/services/BookQueryServiceImpl.java:30` · `course/services/CourseQueryServiceImpl.java:36` · `enrollment/services/EnrollmentQueryServiceImpl.java:29` · `student/services/StudentQueryServiceImpl.java:124`

CUM E ACUM: aplicație pornită pe o bază goală, primele patru cereri ale unui client:

```
GET /api/students    -> HTTP 500
GET /api/courses     -> HTTP 500
GET /api/books       -> HTTP 500
GET /api/enrollments -> HTTP 500
```

În log, cele patru excepții:

```
academy.hub.app.student.exceptions.NoStudentsFound: No students found
academy.hub.app.course.exceptions.NoCourseFound: No course found
academy.hub.app.book.exceptions.NoBookFound: No book found
academy.hub.app.enrollment.exceptions.NoEnrollmentsFound: No Enrollments found
```

Cu date în bază (am rulat `seed()`), trei din patru dau 200. `GET /api/books` dă 500 și atunci, pentru că nicio carte nu poate fi creată (vezi B5).

DE CE: „nu există niciun student” nu e o eroare — e un răspuns. Eroarea înseamnă „cererea ta n-a putut fi onorată”; aici a fost onorată perfect, iar rezultatul corect este `[]`. Diferența se vede din partea clientului: un frontend care primește `200 []` afișează „nicio înregistrare”; unul care primește `500` afișează „a picat serverul” și nu are cum să distingă între „bază goală” și „bază căzută”.

Există un singur caz în care lipsa e într-adevăr eroare: când ceri **un anume** lucru după id (`GET /api/students/{id}` cu un id inexistent) — acolo 404 e corect, pentru că resursa pe care ai numit-o nu există. Lista nu numește nimic, deci nu poate lipsi.

FIX: scoți `throw` din cele patru metode de listare și returnezi lista așa cum vine. Excepțiile `NoStudentsFound` / `NoCourseFound` / `NoBookFound` / `NoEnrollmentsFound` rămân nefolosite — se șterg.

Același bug, în metodele neexpuse încă: `StudentQueryServiceImpl.java:29`, `:98`, `:105`, `:112` · `CourseQueryServiceImpl.java:44`, `:58`, `:80` · `BookQueryServiceImpl.java:36`, `:44`. Sunt 11 locuri cu aceeași regulă greșită; de asta îl numesc bug de regulă, nu de linie.

B2 — Niciun `@RestControllerAdvice` : șase excepții diferite, toate 500 cu corp gol

Nu există niciun `@RestControllerAdvice` în proiect (căutat în tot `src/`). A treia rundă la rând cu aceeași constatare.

CUM E ACUM: șase situații complet diferite, dovedite prin cereri reale, răspund identic:

Cerere	Ce ar trebui	Ce primești
POST /api/enrollments cu pereche deja existentă	409 Conflict	500
POST /api/enrollments cu studentId inexistent	404 Not Found	500
PUT /api/enrollments/{id} cu id inexistent	404 Not Found	500
DELETE /api/enrollments cu pereche inexistentă	404 Not Found	500
PUT /api/enrollments/{id} cu body {}	400 Bad Request	500
GET /api/books pe bază goală	200 []	500

Corpul răspunsului, identic la toate șase:

```
{"timestamp":"...", "status":500, "error":"Internal Server Error",
  "path":"/api/enrollments"}
```

Mesajele pe care le-ai scris în `ExceptionHandlerConstants` („Student already enrolled in this course”, „Enrollment not found”) **nu ajung niciodată la client**. Sunt doar în log-ul tău.

DE CE: Spring MVC nu știe nimic despre `StudentAlreadyEnrolledInThisCourse`. Când excepția iese din controller, DispatcherServlet întreabă pe rând fiecare `HandlerExceptionHandler` dacă o recunoaște. `@ExceptionHandler` -ele tale ar fi fost consultate primele — dar nu există niciunul. Următorul, `ResponseStatusExceptionHandler`, caută `@ResponseStatus` pe clasa excepției — nu ai. Ultimul, `DefaultHandlerExceptionHandler`, cunoaște doar excepțiile proprii ale lui Spring. Niciunul nu o revendică, deci cererea e redirectată intern spre `/error`, iar `BasicErrorController` o tratează ca pe orice eroare necunoscută: 500, fără mesaj (mesajul e ascuns implicit, `server.error.include-message=never`).

Al șaptelea caz e cel care arată cel mai clar de ce contează: la `PUT` cu body `{}` iese `InvalidDataAccessApiUsageException: The given id must not be null` — o excepție internă Spring Data, scursă spre client ca 500. Fără advice, orice defect intern al aplicației arată pentru client exact la fel ca o greșeală a lui.

FIX: o singură clasă `@RestControllerAdvice` cu câte un `@ExceptionHandler` pe familie (not-found → 404, conflict → 409, validare → 400), care întoarce un corp cu `message`.

B3 — **PUT** pe o înscriere cu aceeași pereche (student, curs) → 500

enrollment/services/EnrollmentCommandServiceImpl.java:79

```
if(enrollmentRepository.findByStudentIdAndCourseId(
    eur.studentId(), eur.courseId()).isPresent()) {
    throw new StudentAlreadyEnrolledInThisCourse();
}
```

CUM E ACUM: reproduș în trei pași —

```
POST /api/enrollments {Rares, Microeconomics}    -> 200, id = 35af910c
PUT  /api/enrollments/35af910c {Rares, Microeconomics} -> 500
```

Adică: nu poți trimite de două ori același **PUT**. Un client care reîncearcă după un timeout, sau un formular din care utilizatorul a schimbat doar o dată și apoi salvează din nou, primește „studentul e deja înscris la cursul ăsta” — despre înscrierea pe care tocmai încerca s-o salveze.

DE CE: verificarea răspunde la întrebarea „există vreo înscriere cu perechea asta?”, dar întrebarea corectă e „există **altă** înscriere cu perechea asta?”. Rândul pe care tocmai îl modifici se găsește singur în căutare. E aceeași formă de bug ca la **changeBudget** din company-management, unde **where d.name** prindea și rândul vecin: filtrul e corect ca sintaxă, dar mulțimea pe care o descrie nu e cea pe care o voiai.

Regula asta nu se poate apăra complet din cod. Tabela **enrollment** nu are încă o constrângere **UNIQUE (student_id, course_id)** — atât timp cât n-o are, două cereri simultane pot trece amândouă de **if** înainte ca vreuna să scrie.

FIX: verificarea exclude rândul curent —

```
enrollmentRepository.findByStudentIdAndCourseId(...)
    .filter(e -> !e.getId().equals(id))
    .ifPresent(e -> { throw new StudentAlreadyEnrolledInThisCourse(); });
```

```
plus @Table(name="enrollment", uniqueConstraints = @UniqueConstraint(
    name="uk_enrollment_student_course", columnNames={"student_id","course_id"})) .
```

B4 — **@Valid** pe **EnrollmentUpdateRequest** și **EnrollmentDeleteRequest** nu verifică nimic

enrollment/dtos/EnrollmentUpdateRequest.java:5 ·
enrollment/dtos/EnrollmentDeleteRequest.java:5

```
public record EnrollmentUpdateRequest(UUID studentId, UUID courseId) {}
public record EnrollmentDeleteRequest(UUID studentId, UUID courseId) {}
```

CUM E ACUM: `PUT /api/enrollments/{id}` cu body `{}` →

```
HTTP 500 · InvalidDataAccessApiUsageException: The given id must not be null
```

DE CE: `@Valid` din controller (`EnrollmentController.java:39`) nu inventează reguli — el doar pornește motorul de validare pe adnotările găsite pe câmpuri. Pe recordul ăsta nu e niciuna, deci motorul nu are ce verifica și lasă cererea să treacă cu `null` pe ambele câmpuri. `null`-ul călătorește nestingherit până în Spring Data, care abia acolo refuză.

Că ai știut regula se vede pe DTO-ul fratele lui: `EnrollmentCreateRequest.java:9,12` are `@NotNull` pe amândouă câmpurile — de aceea `POST` fără `studentId` răspunde corect **400**, dovedit în aceeași rundă de probe. Aceeași aplicație, același tip de câmp, două comportamente diferite, pentru că regula a fost pusă pe DTO-ul semnalat, nu pe toate trei.

FIX: `@NotNull` pe cele patru câmpuri din `EnrollmentUpdateRequest` și `EnrollmentDeleteRequest`.

B5 — Modulul `book` e blocat: nicio carte nu poate fi creată (runda 2, B4 — NEATINS)

`book/services/BookCommandServiceImpl.java:29–48` + `book/models/Book.java:36–38`

CUM E ACUM: reprodus azi, din nou:

```
PROBE-FAIL [createBook] -> DataIntegrityViolationException | root:
JdbcSQLIntegrityConstraintViolationException:
NULL not allowed for column "STUDENT_ID";
insert into book (created_at,name,student_id,id) values (?,?,,?)
```

DE CE: `Book.student` e `@ManyToOne(optional = false)` cu `@JoinColumn(nullable = false)`, dar `BookCreateRequest` are un singur câmp — `name`. Serviciul construiește `new Book(name, createdAt)` și salvează. Coloana `student_id` rămâne `null`, iar baza refuză rândul. Nu e o eroare care apare „uneori”: e singurul rezultat posibil al metodei.

Consecința se vede în lanț: cum nicio carte nu poate intra în bază, tabela `book` e mereu goală, deci `GET /api/books` trece prin `throw new NoBookFound()` (B1) și răspunde 500 — la orice cerere, pentru totdeauna. Două bug-uri diferite care se acoperă unul pe altul: îl repari pe B1 și tot nu vezi nicio carte, îl repari pe B5 și tot primești 500 pe listă.

FIX: cartea nu există fără proprietar, deci cererea trebuie să-l poarte: `BookCreateRequest(UUID studentId, String name)` → încarci studentul → `student.addBook(book)` → salvezi. Decizia de la runda 1 (`Student` =

aggregate root) rămâne valabilă.

B6 — Trei servicii de scriere din patru nu sunt accesibile din API

```
student/controllers/StudentController.java · course/controllers/CourseController.java ·  
book/controllers/BookController.java
```

CUM E ACUM:

```
POST /api/students -> HTTP 405 Method Not Allowed  
POST /api/courses -> HTTP 405 Method Not Allowed  
POST /api/books -> HTTP 405 Method Not Allowed
```

Cele trei controllere au fiecare un singur `@GetMapping`. `StudentCommandService`, `CourseCommandService` și `BookCommandService` există, sunt scrise, sunt înregistrate ca bean-uri — și nu le cheamă nimeni. Singurul loc din care erau chemate, `Runner.run()` (`Runner.java:75-85`), are acum toate liniile comentate.

DE CE: 405, nu 404, pentru că Spring a găsit ruta `/api/students` (o mapează `@RequestMapping` de pe clasă) dar nu și metoda HTTP cerută — deci îți spune „calea există, verbul nu”. E util de știut la diagnostic: 404 = n-am ruta, 405 = am ruta, n-am verbul.

E a treia rundă cu cod de scriere inaccesibil (la runda 1 era `EmployeeCommandServiceImpl` în company-management, exact aceeași formă). Un serviciu pe care nu-l cheamă nimeni nu e cod pregătit pentru mai târziu — e cod care se strică în tăcere, pentru că nimic nu-l mai execută ca să-i observi defectele. B5 e dovada: bug-ul din `createBook` a supraviețuit o rundă întreagă tocmai pentru că nimeni nu mai chema metoda.

FIX: POST / PUT / DELETE pe cele trei controllere, câte unul pentru fiecare metodă existentă în command service-uri.

🟡 Importante

M1 — un graf cu două colecții multiplică rândurile citite din bază.

```
student/repository/StudentRepository.java:47 :
```

```
@EntityGraph(attributePaths = {"enrollments", "books", "enrollments.course"})  
List<Student> findAll();
```

Măsurat azi, cu 6 studenți, 12 înscrieri și 2 cărți: SQL-ul generat întoarce **15 rânduri pentru 6 studenți**. Răspunsul JSON e corect (Hibernate elimină duplicatele pentru că ai folosit `Set`, nu `List`), iar numărul de

interogări rămâne 1 — de asta e invizibil acum. Costul crește însă înmulțit: un student cu 10 cărți și 10 cursuri produce singur 100 de rânduri. Regula: un singur `join fetch` pe colecție per interogare; a doua colecție se aduce separat. (Dacă vreodată schimbi `Set` pe `List`, aceeași linie se transformă în `MultipleBagFetchException` la pornire.)

M2 — fiecare verificare rulează interogarea de două ori. Tipar prezent în 17 locuri:

`CourseQueryServiceImpl.java:44,58,66,74,80` .

`StudentQueryServiceImpl.java:29,66,74,82,98,105,112` . `BookQueryServiceImpl.java:36,44` .

Semnalat și la runda 2 (M4), neatins.

```
if(courseRepository.findById(id).isEmpty()){ throw new NoCourseFound(); }  
return courseRepository.findById(id).orElseThrow(); // al doilea SELECT
```

Se rezolvă cu o variabilă locală, sau direct `findById(id).orElseThrow(NoCourseFound::new)` .

M3 — verificări `== null` care nu se pot declanșa. `StudentQueryServiceImpl.java:36,43,50,58` .

`BookQueryServiceImpl.java:36,44` . Un derived query care întoarce `List` întoarce listă goală, niciodată `null` . Semnalat la runda 2 (M5), neatins.

M4 — `POST` răspunde 200, nu 201. `EnrollmentController.java:33` . La creare se întoarce `201 Created` cu antetul `Location` spre resursa nouă.

M5 — `DELETE /api/enrollments` primește corpul cererii, nu id-ul din cale.

`EnrollmentController.java:43-46` . Merge (l-am rulat, 200), dar multe proxy-uri și clienți HTTP aruncă corpul unui DELETE. Forma așteptată: `DELETE /api/enrollments/{id}` .

M6 — `Enrollment.equals` crapă pe obiecte fără student sau curs. `Enrollment.java:47` . Semnalat la runda 2 (B6), neatins. Compară `student.getId()` fără să verifice `student` .

M7 — `deleteStudent` ignoră emailul din request. `StudentCommandServiceImpl.java:54-62` . Semnalat la runda 2 (B7), neatins.

M8 — `updateStudent` nu verifică unicitatea emailului (runda 2, M2, neatins) și **`deleteStudent` aruncă `EmailNotFound` pentru un id inexistent** (runda 2, M3, neatins).

M9 — `@Transactional` lipsă pe `CourseCommandServiceImpl` și `EnrollmentCommandServiceImpl` (runda 2, M6, neatins). Acum contează mai mult decât atunci: metodele sunt expuse prin HTTP, deci „verific, apoi scriu” poate fi întrerupt între cele două tranzacții de o a doua cerere.

M10 — structura cerută la runde 1 și 2, neatinsă. `ExceptionConstants` există în patru pachete cu același nume . `services/interfaces/` în loc de `services/` + `services/impl/` . plural inconsistent: `models` , `dtos` , `services` , `controllers` , dar `repository` la singular.

Cleanups

- **C1** — `Enrollment.java:9` : `import org.springframework.cglib.core.Local;` — nefolosit, ales din auto-complete. Semnalat la runda 2 (C1).
 - **C2** — `StudentSummary.java:3` : un DTO importă `CourseRepository` . Semnalat (C2).
 - **C3** — `BookQueryServiceImpl.java:14` e `@Component` ; restul sunt `@Service` . Semnalat (C3).
 - **C4** — `EnrollmentCommandServiceImpl.java:29` : constructorul primește `studentRepository` și `courseRepository` și nu le atribuie nicăieri. Spring le injectează degeaba.
 - **C5** — `deletebook` , `updatebook` cu literă mică. Semnalat la runda 2 (C8).
 - **C6** — `Runner.java:75–85` : toate apelurile sunt comentate. Clasa are peste 400 de linii care nu se mai execută. Odată ce există controllere, locul lor e la gunoi sau într-un seeder pornit doar pe un profil (`@Profile("demo")`).
 - **C7** — `BookController.java:5` : `import ...book.models.Book;` nefolosit.
-

Before / After (doar pentru 🚫)

#	Acum	Corect
B1	<code>if (books.isEmpty()) { throw new NoBookFound(); }</code>	<code>return books;</code> — lista goală e <code>200 []</code>
B2	nicio clasă de advice	<code>@RestControllerAdvice</code> cu <code>@ExceptionHandler</code> pe familie: not-found → 404, conflict → 409, validare → 400
B3	<code>findByStudentIdAndCourseId(...).isPresent()</code>	<code>.filter(e -> !e.getId().equals(id)).ifPresent(...)</code> + <code>UNIQUE (student_id, course_id)</code> pe tabelă
B4	<code>record EnrollmentUpdateRequest(UUID studentId, UUID courseId)</code>	<code>@NotNull</code> pe fiecare câmp, ca la <code>EnrollmentCreateRequest</code>
B5	<code>new Book(name, createdAt)</code> salvat fără student	<code>BookCreateRequest(UUID studentId, String name)</code> → <code>student.addBook(book)</code> → salvezi studentul
B6	controllere doar cu <code>@GetMapping</code>	<code>@PostMapping</code> , <code>@PutMapping</code> , <code>@DeleteMapping</code> care cheamă command service-urile existente

Q&A — de răspuns înainte de runda 4

- B1 + B2.** `GET /api/books` pe bază goală răspunde 500. Sunt de fapt două defecte suprapuse acolo. Care e fiecare, și dacă l-ai repara pe unul singur, ce ar răspunde endpoint-ul atunci?
- B3.** Verificarea ta de duplicat e corectă la `create` și greșită la `update`, deși e aceeași linie de cod. Ce s-a schimbat între cele două situații — și de ce o constrângere `UNIQUE` în tabelă ar fi apărut regula în amândouă, fără nicio linie de Java?
- B6.** `POST /api/students` răspunde 405, nu 404. Ce înseamnă diferența asta despre ce a găsit și ce n-a găsit Spring — și cum ai folosi-o data viitoare ca să-ți dai seama din prima dacă ai greșit calea sau ai uitat metoda?

Runda 2 — 2026-08-19 — sha 8b84a6d

- ✓ B1 `Course.enrollments` neinițializat → NPE — **închis**
- ✓ B2 `StudentSummary.getId()` `Long` peste `UUID` — **închis**
- ✓ B3 `@EntityGraph` către un graf inexistent — **închis**
- ● B4 `createBook` nu poate reuși niciodată — **NEATINS** (runda 3, B5)
- ● B5 `updateEnrollment` duplică perechea — verificarea a fost adăugată, dar prinde și rândul propriu (runda 3, B3); constrângerea `UNIQUE` pe tabelă tot lipsește
- ● B6 `Enrollment.equals` crapă fără student/curs — **NEATINS** (runda 3, M6)
- ● B7 `deleteStudent` ignoră emailul din request — **NEATINS** (runda 3, M7)
- M1 `updateCourse` scrie orbește · M2 email duplicat la update · M3 excepție greșită · M4 query dublu · M5 `== null` imposibil · M6 `@Transactional` lipsă · M7 factory · M8 `Enrollment` incomplet — **toate NEATINSE** (runda 3, M2–M9)
- C1–C8 — neatins, cu excepția celor dispărute odată cu refacerea fișierelor

Runda 1 — 2026-08-05

- ✓ `spring-boot-starter-web` + `controllers/` — **închise la runda 3**
- ● `@RestControllerAdvice` — **NEATINS de trei runde** (runda 3, B2)
- ● `services/interfaces/` → `services/` + `services/impl/` — **NEATINS**
- ● `ExceptionConstants` duplicat (acum în 4 pachete) — **NEATINS**
- ● plural inconsistent `repository` vs `models / dtos / services` — **NEATINS**
- Decizia de arhitectură: `Student` = aggregate root, `book/` fără `services/` proprii — contrazisă de `BookCommandService`, de reluat explicit

